ServiX

Gestión de clientes, servicios y atención con IA

DOCUMENTACIÓN DE FIREBASE

Documentación de Firebase en ServiX

Modelo de datos, reglas de seguridad, operaciones, listeners e índices del backend gestionado

Proyecto	ServiX — Gestión de clientes, servicios y atención con IA
Código del documento	DOC-07
Versión	1.0
Fecha	01/11/2026
Estándar de referencia	Documentación técnica del proyecto; modelo de datos descrito según ISO/IEC/IEEE 1016
Institución	Universidad de San Buenaventura
Asignatura	Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Nicolas Quiroz Gomez	Product Owner
Juan Sebastian Garzon Suarez	Scrum Master
Juan Esteban Olaya Muñoz	Developer

Control del documento

Información general

Título	Documentación de Firebase en ServiX
Código	DOC-07
Versión	1.0
Fecha de emisión	01/11/2026
Estado	Versión 1.0 emitida; aprobaciones pendientes
Proyecto	ServiX
Autor	Equipo ServiX
Clasificación	Uso académico; sin información confidencial

Historial de revisiones

Versión	Fecha	Descripción del cambio	Autor
0.9	01/11/2026	Borrador inicial en formato plano (archivado como respaldo v1).	Equipo ServiX
1.0	01/11/2026	Rediseño completo: portada, control de versiones, numeración, figuras y tablas rotuladas, mayor profundidad.	Equipo ServiX

Aprobaciones

Rol	Nombre	Fecha	Firma / estado
Elaboró	Juan Esteban Olaya Muñoz (Developer)	Pendiente	Pendiente
Revisó	Juan Sebastian Garzon Suarez (Scrum Master)	Pendiente	Pendiente
Aprobó	Nicolas Quiroz Gomez (Product Owner)	Pendiente	Pendiente

Las aprobaciones se encuentran **pendientes**: no se registran firmas ni fechas de aprobación hasta que los responsables las emitan.

Tabla de contenido

1 Introducción **5**
2 Modelo de datos **6**
2.1 Diccionario de datos 7
2.2 Ejemplos de documentos (datos ficticios) 10
3 Reglas de seguridad **12**
3.1 Funciones auxiliares 12
3.2 Reglas por ruta 12
3.3 Despliegue de las reglas 13
4 Operaciones de datos **14**
4.1 Operaciones por repositorio 14
4.2 Ciclo de vida de los listeners 14
5 Índices **15**
6 Costos, cuotas y buenas prácticas **16**
7 Verificación pendiente **17**

Lista de figuras

Figura 1. Modelo de datos de ServiX en Cloud Firestore 6

Lista de tablas

Tabla 1. Servicios de Firebase utilizados 5
Tabla 2. Estructura de colecciones 6
Tabla 3. Diccionario de datos de usuarios/{uid} (UsuarioDto) 7
Tabla 4. Diccionario de datos de usuarios/{uid}/clientes/{id} (ClienteDto) 7
Tabla 5. Diccionario de datos de usuarios/{uid}/servicios/{id} (ServicioDto) 7
Tabla 6. Diccionario de datos de usuarios/{uid}/notificaciones/{id} (NotificacionDto) 8
Tabla 7. Diccionario de datos de usuarios/{uid}/actividades/{id} (ActividadDto) 8
Tabla 8. Diccionario de datos de usuarios/{uid}/configuracion/app_config (AppConfigDto) 9
Tabla 9. Diccionario de datos de usuarios/{uid}/configuracion/user_profile (UserProfileDto) 9
Tabla 10. Diccionario de datos de solicitudes_atencion/{id} (SolicitudAtencionDto) 9
Tabla 11. Funciones auxiliares de las reglas 12
Tabla 12. Reglas por ruta explicadas 12
Tabla 13. Operaciones CRUD y listeners por repositorio 14
Tabla 14. Consultas y su índice 15
Tabla 15. Elementos de Firebase pendientes de verificar 17

1 Introducción

ServiX no tiene un servidor propio: usa tres servicios de Firebase del proyecto `servix-453fb`. Este documento describe cómo se usa cada uno, con énfasis en Cloud Firestore: estructura de datos, diccionario de campos, reglas de seguridad, operaciones de lectura y escritura, escuchas en tiempo real e índices.

Tabla 1. Servicios de Firebase utilizados

Servicio	Uso en ServiX	SDK	Estado de verificación
Authentication	Registro e inicio de sesión con correo y contraseña; restablecimiento de contraseña.	`firebase-auth`	Probado solo con repositorios simulados; real: no ejecutado.
Cloud Firestore	Datos de negocio por usuario y solicitudes de atención.	`firebase-firestore`	Reglas desplegadas; acceso real: no ejecutado.
Firebase AI Logic	Respuestas del asistente con Gemini.	`firebase-ai`	No ejecutado (véase DOC-08).
App Check	Dependencia incluida; sin proveedor inicializado.	`firebase-appcheck-recaptcha`	No activo.

Nota. Hosting
El proyecto no usa Firebase Hosting ni Cloud Functions. `firebase.json` solo declara las reglas de Firestore.

2 Modelo de datos

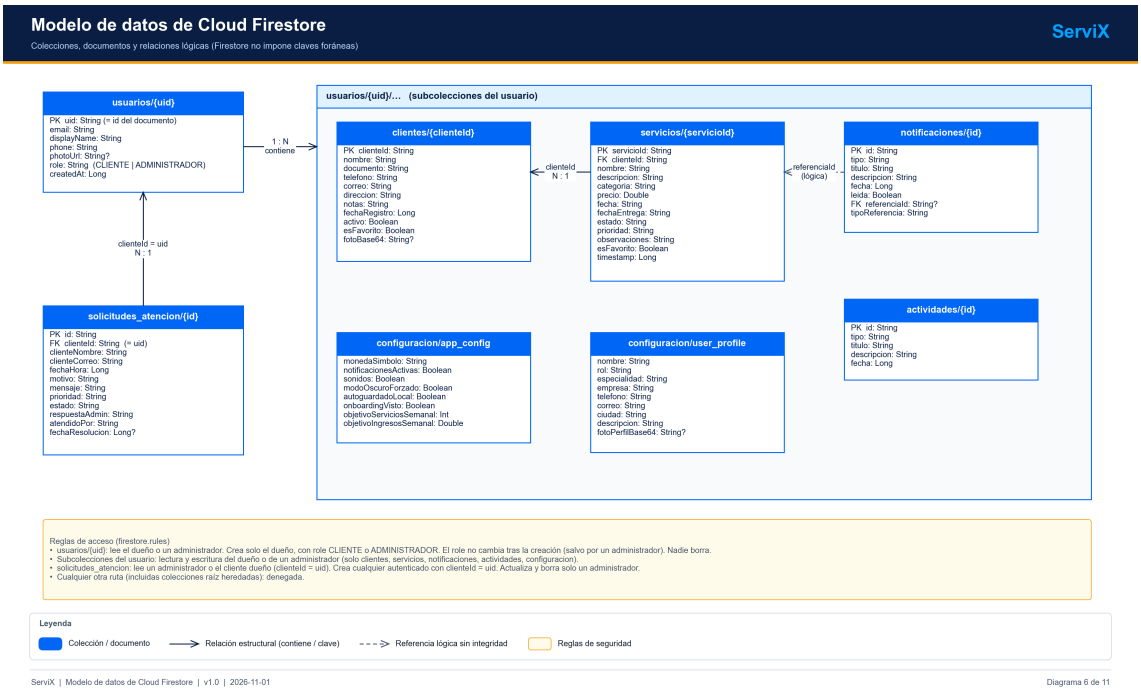


Figura 1. Modelo de datos de ServiX en Cloud Firestore

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Cada usuario tiene un documento de perfil en `usuarios/{uid}` y cinco subcolecciones. Las solicitudes de atención viven en una colección raíz porque deben ser visibles tanto para el cliente que las crea como para el administrador que las atiende.

Tabla 2. Estructura de colecciones

Ruta	Tipo	Descripción
`usuarios`	Colección raíz	Un documento por cuenta; la clave es el `uid` de Authentication.
`usuarios/{uid}/clientes`	Subcolección	Clientes del negocio.
`usuarios/{uid}/servicios`	Subcolección	Órdenes de servicio; se vinculan con `clienteId`.
`usuarios/{uid}/notificaciones`	Subcolección	Notificaciones internas.
`usuarios/{uid}/actividades`	Subcolección	Historial de actividades.
`usuarios/{uid}/configuracion`	Subcolección	Dos documentos: `app_config` y `user_profile`.
`solicitudes_atencion`	Colección raíz	Casos escalados desde el chat.

2.1 Diccionario de datos

Tabla 3. Diccionario de datos de usuarios/{uid} (UsuarioDto)

Campo	Tipo	Descripción	Valor por defecto
`uid`	string	Identificador de la cuenta.	""
`email`	string	Correo de la cuenta.	""
`displayName`	string	Nombre visible.	""
`phone`	string	Teléfono (puede estar vacío).	""
`photoUrl`	string, nulo	Dirección de la foto.	nulo
`role`	string	`CLIENTE` o `ADMINISTRADOR`. Inmutable para el dueño.	`CLIENTE`
`createdAt`	número (milisegundos)	Fecha de creación.	hora actual

Tabla 4. Diccionario de datos de usuarios/{uid}/clientes/{id} (ClienteDto)

Campo	Tipo	Descripción	Valor por defecto
`clienteId`	string	Identificador del cliente.	""
`nombre`	string	Nombre completo.	""
`documento`	string	Documento de identidad.	""
`telefono`	string	Teléfono.	""
`correo`	string	Correo de contacto.	""
`direccion`	string	Dirección.	""
`notas`	string	Observaciones.	""
`fechaRegistro`	número (ms)	Fecha de alta.	hora actual
`activo`	booleano	Cliente activo.	verdadero
`esFavorito`	booleano	Marcado como favorito.	falso
`fotoBase64`	string, nulo	Foto codificada en Base64.	nulo

Tabla 5. Diccionario de datos de usuarios/{uid}/servicios/{id} (ServicioDto)

Campo	Tipo	Descripción	Valor por defecto
`servicioId`	string	Identificador del servicio.	""
`clienteId`	string	Cliente al que pertenece.	""
`nombre`	string	Nombre del servicio.	""
`descripcion`	string	Descripción.	""
`categoria`	string	Categoría.	`General`
`precio`	número decimal	Precio informativo.	0,0

Campo	Tipo	Descripción	Valor por defecto
fecha	string	Fecha programada (texto).	""
fechaEntrega	string	Fecha de entrega (texto).	""
estado	string	PENDIENTE, EN_PROCESO, COMPLETADO o CANCELADO.	PENDIENTE
prioridad	string	BAJA, MEDIA, ALTA o URGENTE.	MEDIA
observaciones	string	Notas.	""
esFavorito	booleano	Favorito.	falso
timestamp	número (ms)	Marca de creación.	hora actual

Tabla 6. Diccionario de datos de usuarios/{uid}/notificaciones/{id} (NotificacionDto)

Campo	Tipo	Descripción	Valor por defecto
id	string	UUID de la notificación.	""
tipo	string	SERVICIO_HOY, SERVICIO_URGENTE, SERVICIO_PENDIENTE, SERVICIO_COMPLETADO, NUEVO_CLIENTE, RECORDATORIO_SEGUIMIENTO, SOLICITUD_ATENCION.	SERVICIO_HOY
titulo	string	Título.	""
descripcion	string	Detalle.	""
fecha	número (ms)	Fecha de generación.	hora actual
leida	booleano	Leída por el usuario.	falso
referenciaId	string, nulo	Identificador del elemento relacionado.	nulo
tipoReferencia	string	servicio, cliente o atencion.	servicio

Tabla 7. Diccionario de datos de usuarios/{uid}/actividades/{id} (ActividadDto)

Campo	Tipo	Descripción	Valor por defecto
id	string	Identificador.	""
tipo	string	CLIENTE_CREADO, CLIENTE_EDITADO, CLIENTE_ELIMINADO, SERVICIO_CREADO, SERVICIO_ACTUALIZADO, SERVICIO_COMPLETADO, SERVICIO_CANCELADO.	SERVICIO_CREADO
titulo	string	Título.	""
descripcion	string	Detalle.	""
fecha	número (ms)	Fecha del evento.	hora actual

Tabla 8. Diccionario de datos de usuarios/{uid}/configuracion/app_config (AppConfigDto)

Campo	Tipo	Descripción	Valor por defecto
monedaSimbolo	string	Símbolo de moneda.	$
notificacionesActivas	booleano	Alertas habilitadas.	verdadero
sonidos	booleano	Sonidos habilitados.	verdadero
modoOscuroForzado	booleano	Fuerza el tema oscuro.	falso
autoguardadoLocal	booleano	Autoguardado.	verdadero
onboardingVisto	booleano	Tutorial visto.	verdadero
objetivoServiciosSemanal	entero	Meta semanal de servicios.	10
objetivoIngresosSemanal	número decimal	Meta semanal de ingresos.	1.200.000

Tabla 9. Diccionario de datos de usuarios/{uid}/configuracion/user_profile (UserProfileDto)

Campo	Tipo	Descripción	Valor por defecto
nombre	string	Nombre del responsable.	valor de ejemplo
rol	string	Cargo o descripción del rol.	valor de ejemplo
especialidad	string	Especialidad.	valor de ejemplo
empresa	string	Nombre del negocio.	valor de ejemplo
telefono	string	Teléfono.	valor de ejemplo
correo	string	Correo del negocio.	valor de ejemplo
ciudad	string	Ciudad.	valor de ejemplo
descripcion	string	Descripción del negocio.	valor de ejemplo
fotoPerfilBase64	string, nulo	Foto en Base64.	nulo

Tabla 10. Diccionario de datos de solicitudes_atencion/{id} (SolicitudAtencionDto)

Campo	Tipo	Descripción	Valor por defecto
id	string	UUID; los primeros 8 caracteres, en mayúsculas, forman el radicado.	""
clienteId	string	uid del cliente que la crea (las reglas lo exigen).	""
clienteNombre	string	Nombre del cliente (hasta 200 caracteres).	""
clienteCorreo	string	Correo del cliente.	""
fechaHora	número (ms)	Fecha de creación.	hora actual
motivo	string	Motivo (hasta 500 caracteres).	""
mensaje	string	Mensaje original (hasta 4000 caracteres).	""
prioridad	string	NORMAL, IMPORTANTE o URGENTE.	NORMAL

Campo	Tipo	Descripción	Valor por defecto
`estado`	string	`PENDIENTE`, `EN_ATENCION` o `RESUELTO`.	`PENDIENTE`
`respuestaAdmin`	string	Respuesta del administrador.	""
`atendidoPor`	string	Nombre del responsable.	""
`fechaResolucion`	número (ms), nulo	Fecha de resolución.	nulo

Nota. Tolerancia a documentos incompletos
Los DTO tienen valores por defecto y los campos de tipo enumerado se convierten con `valueOf` dentro de un `try/catch` que recurre al valor inicial. Un documento con un estado o prioridad desconocidos no rompe la aplicación.

2.2 Ejemplos de documentos (datos ficticios)

Ejemplo de documento de perfil

```
// usuarios/{uid}
{
  "uid": "uid-de-ejemplo-001",
  "email": "(correo ficticio)",
  "displayName": "Cliente de Ejemplo",
  "phone": "",
  "role": "CLIENTE",
  "createdAt": 1793500000000
}
```

Ejemplo de documento de servicio

```
// usuarios/{uid}/servicios/{id}
{
  "servicioId": "srv-0001",
  "clienteId": "cli-0001",
  "nombre": "Mantenimiento preventivo de aire acondicionado",
  "categoria": "Climatización",
  "precio": 180000.0,
  "fecha": "05/11/2026",
  "estado": "EN_PROCESO",
  "prioridad": "ALTA",
  "esFavorito": false,
  "timestamp": 1793500100000
}
```

Ejemplo de solicitud de atención

```
// solicitudes_atencion/{id}
{
  "id": "9f3c1a7e-0000-0000-0000-000000000000",
  "clienteId": "uid-de-ejemplo-001",
  "clienteNombre": "Cliente de Ejemplo",
  "motivo": "Queja sobre la atención o calidad del servicio",
  "mensaje": "Tengo una queja por el retraso de mi orden",
  "prioridad": "IMPORTANTE",
  "estado": "PENDIENTE",
  "respuestaAdmin": "",
  "atendidoPor": "",
  "fechaResolucion": null
}
// Radicado mostrado al usuario: 9F3C1A7E
```

3 Reglas de seguridad

Las reglas están en `firestore.rules` y se despliegan con la Firebase CLI. Siguen el principio de mínimo privilegio: lo que no se permite explícitamente se deniega.

3.1 Funciones auxiliares

Tabla 11. Funciones auxiliares de las reglas

Función	Qué hace
`isSignedIn()`	Verdadero si la petición lleva un usuario autenticado.
`isOwner(uid)`	Verdadero si el usuario autenticado es el propietario del `uid`.
`isAdmin()`	Lee el campo `role` de `usuarios/{auth.uid}` y acepta `ADMINISTRADOR`, `ADMIN`, `VENDEDOR` y `ADMIN_VENDEDOR` (valores heredados).
`isValidRole(role)`	Solo admite `CLIENTE` o `ADMINISTRADOR` al crear el perfil.
`isShortString(valor, max)`	Verifica que el valor sea texto con longitud máxima.

3.2 Reglas por ruta

Tabla 12. Reglas por ruta explicadas

Ruta	Operación	Condición	Efecto
`usuarios/{userId}`	Lectura	Dueño o administrador	Cada usuario lee su perfil.
	Creación	Dueño y `role` válido	Autoregistro; permite elegir `ADMINISTRADOR` (riesgo R-01).
	Actualización	Dueño sin cambiar `role`, o administrador	El rol es inmutable para el dueño.
	Borrado	Nunca	Los perfiles no se eliminan desde la aplicación.
`usuarios/{userId}/{sub}/{doc}`	Lectura y escritura	(Dueño o administrador) y la subcolección está en la lista	Solo `clientes`, `servicios`, `notificaciones`, `actividades`, `configuracion`.
`solicitudes_atencion/{id}`	Lectura	Administrador, o `clienteId == auth.uid`	El cliente ve solo las suyas.
	Creación	Autenticado, `clienteId == auth.uid` y límites de longitud	Evita suplantar a otro cliente y limita el tamaño.
	Actualización y borrado	Administrador	Solo el administrador toma, responde y resuelve.
`/{document=**}`	Cualquiera	Nunca	Deniega lo no declarado.

> **Advertencia. Limitaciones de las reglas**
> (1) Cualquiera puede crear su perfil con rol `ADMINISTRADOR`. (2) `isAdmin()` hace una lectura adicional por petición (consume cuota). (3) Las reglas no se han probado con el emulador de reglas (CP-042). (4) Las reglas permiten al dueño de un árbol escribir en él aunque su rol sea cliente; la restricción de funciones por rol la aplica solo la interfaz.

3.3 Despliegue de las reglas

Despliegue

```
firebase login
firebase use servix-453fb
firebase deploy --only firestore:rules
```

Tras desplegar, compruebe la fecha de publicación en la consola de Firebase (Firestore → Reglas) y pruebe una lectura no autorizada en el simulador de reglas de la consola.

4 Operaciones de datos

4.1 Operaciones por repositorio

Tabla 13. Operaciones CRUD y listeners por repositorio

Repositorio	Crear	Leer / escuchar	Actualizar	Eliminar
Auth (`usuarios`)	`set` del perfil tras registrar o iniciar sesión	`get` del rol; `AuthStateListener`	`set` del perfil al iniciar sesión	No
Clientes	`set` con identificador generado	`addSnapshotListener` de la subcolección	`set` del documento completo; favorito	`delete` del cliente y de sus servicios (consulta por `clienteId`)
Servicios	`set` con identificador generado	`addSnapshotListener`	`set` del documento; estado; favorito	`delete` del documento
Notificaciones	`set` por UUID	`addSnapshotListener`	`update("leida", true)` individual o por lote	`delete`
Actividades	`set` con identificador generado	`addSnapshotListener`	No	No
Configuración	`set` de `app_config` y `user_profile`	Escucha de cada documento	`set`	No
Atención	`set` por UUID	Escucha filtrada por `clienteId` (cliente) o completa (administrador)	`set` (tomar, responder, resolver)	No

4.2 Ciclo de vida de los listeners

Cada repositorio abre su escucha al conocer el `uid` del usuario (mediante `AuthStateListener`) y la cancela al cerrar sesión o al cambiar de usuario. La interfaz observa los flujos `StateFlow` que los repositorios actualizan con cada instantánea. Los errores de la escucha se registran en el log y el último estado conocido se conserva.

Las escrituras se ejecutan en el ámbito de corrutinas del repositorio y no esperan confirmación del servidor para actualizar la interfaz: Firestore aplica el cambio localmente de inmediato (comportamiento por defecto del SDK) y lo sincroniza cuando hay conexión.

5 Índices

Todas las consultas del código usan igualdad sobre un único campo (`clienteId` en `servicios` y en `solicitudes_atencion`, y `leida` en `notificaciones`), que Firestore resuelve con índices de campo único automáticos. Por tanto el proyecto **no define índices compuestos** y no existe `firestore.indexes.json`.

Tabla 14. Consultas y su índice

Colección	Consulta	Índice requerido
`usuarios/{uid}/servicios`	`whereEqualTo("clienteId", id)`	Campo único automático
`usuarios/{uid}/notificaciones`	`whereEqualTo("leida", false)`	Campo único automático
`solicitudes_atencion`	`whereEqualTo("clienteId", uid)`	Campo único automático

Nota. Si se añade ordenamiento
Una consulta que combine un filtro de igualdad con `orderBy` en otro campo requerirá un índice compuesto; Firestore devuelve en ese caso un enlace para crearlo.

6 Costos, cuotas y buenas prácticas

- Cada lectura de `isAdmin()` en las reglas cuenta como lectura de documento; vigilar el consumo.
- Las escuchas en tiempo real facturan por documento leído; evitar listeners redundantes.
- Restringir la clave de API del proyecto por paquete y huella SHA-1 (pendiente).
- Registrar App Check para que solo la aplicación legítima use Firestore y la IA (pendiente).
- No almacenar contraseñas, tokens ni datos sensibles innecesarios en los documentos; las fotos en Base64 aumentan el tamaño de los documentos.

7 Verificación pendiente

Tabla 15. Elementos de Firebase pendientes de verificar

Elemento	Caso	Estado
Registro e inicio de sesión reales y creación de `usuarios/{uid}`	CP-033	No ejecutado
Gestión de clientes y servicios sobre Firestore	CP-035, CP-036	No ejecutado
Aislamiento: el cliente solo ve sus solicitudes	CP-040	No ejecutado
Reglas deniegan accesos no autorizados	CP-042	No ejecutado
Notificación de solicitud de atención recibida por el administrador (DT-06)	CP-041	No ejecutado